

Resumen Teorico – Practico 6

Testing basado en sintaxis, mutacion y fuzzing

Testing de Software 2024

Índice

Introduccion	3
1. Testing basado en sintaxis: vista general	3
1.1. Cuatro estructuras para modelar software	3
1.2. Por que sintaxis	4
2. Gramaticas BNF	4
2.1. Reconocer vs. generar	5
3. Criterios de cobertura sobre gramaticas	5
3.1. Cadenas mutadas vs. gramaticas mutadas	6
4. Mutacion sobre programas	6
4.1. Tipos de mutantes	6
4.2. Mutation Score	7
4.3. Operadores tipicos	7
4.4. Mutacion fuerte vs. debil	8
4.5. Costo y por que vale la pena	8
5. Fuzzing	8
5.1. Idea general	8
5.2. Random fuzzing	8
5.3. Mutation fuzzing	9
5.4. Random vs. mutation: el tradeoff	9
5.5. El oraculo en fuzzing	9
5.6. Seguridad	10

6. Pitest en la practica	10
6.1. Que hace	10
6.2. Configuracion tipica (pom.xml)	10
6.3. Ejecucion	10
6.4. Como leer el reporte	11
7. Comparacion con criterios anteriores	11
8. Glosario	11
Conclusion	12

Introduccion

Este resumen sintetiza el material teorico asociado al TP6:

- *Introduction to Software Testing* (Ammann & Offutt, 2da edicion), Capitulo 9: **Syntax-Based Testing**.
- *Notas 10: Testing basado en sintaxis* (V. Bengolea).
- *Notas 11: Testing basado en mutacion* (V. Bengolea).

Idea general. Muchos artefactos del software (programas, inputs, mensajes de protocolo, consultas SQL, archivos de configuracion) responden a una *sintaxis estricta*, descrita tipicamente por una gramatica BNF. El **testing basado en sintaxis** usa esa estructura como modelo y deriva tests de dos formas complementarias:

- **Cubriendo** la gramatica: generar cadenas que ejerciten todas las producciones, todos los simbolos terminales, todos los pares de producciones, etc.
- **Violando** la gramatica: mutarla o mutar las cadenas para generar tests *invalidos* que sirven como prueba de robustez.

La aplicacion mas conocida es el **testing de mutacion**, donde lo que se muta es el *propio programa* y la suite debe ser capaz de distinguir el programa original de cada variante. El TP6 cierra con **fuzzing**, que es la realizacion practica de generacion aleatoria/sintactica de inputs.

1. Testing basado en sintaxis: vista general

1.1. Cuatro estructuras para modelar software

Recordando la columna vertebral del libro, hay cuatro grandes formas de modelar software para generar tests:

1. **Espacio de inputs** (TP2): particionar el dominio.
2. **Grafos** (TP4): CFG, FSM, statecharts.
3. **Logica** (TP5): predicados y clausulas.
4. **Sintaxis** (TP6): gramaticas y derivaciones.

Idea clave

La sintaxis se aplica a cuatro fuentes típicas: *codigo*, *modelos* (UML, diagramas), *inputs* permitidos por el programa, y *tests de integracion* (mensajes entre componentes).

1.2. Por que sintaxis

- Cuando hay una gramatica explicita, la generacion de tests es *mecanica*: se puede derivar automaticamente un conjunto enorme de cadenas validas.
- Cuando se quiere testear el *manejo de errores*, se mutan las cadenas (o la gramatica) y se observa que el sistema rechace o reaccione adecuadamente.
- Cuando se quiere medir la *calidad de una suite*, se mutan los programas (testing de mutacion) y se pide que la suite distinga el original del mutante.

2. Gramaticas BNF

Definicion

Una **gramatica BNF** (Backus-Naur Form) esta dada por:

- un conjunto de *simbolos no terminales* N ,
- un conjunto de *simbolos terminales* T disjunto de N ,
- un *simbolo inicial* $S \in N$,
- un conjunto de *producciones* de la forma $nt ::= \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_k$, donde cada α_i es una secuencia de simbolos en $N \cup T$.

El *lenguaje* de la gramatica es el conjunto de cadenas de terminales obtenibles derivando desde S por sustituciones sucesivas.

Notacion auxiliar.

- $*$ (estrella de Kleene): cero o mas repeticiones.
- $+$: una o mas repeticiones.
- $?$ o $[]$: opcional.
- $|$: alternativa.

Ejemplo

Gramatica para un stream de acciones:

```
Stream ::= accion*
accion ::= accG | accB
accG    ::= "G" s n
accB    ::= "B" t n
s       ::= digito^{1..3}
t       ::= digito^{1..3}
n       ::= digito^2 "." digito^2 "." digito^2
digito  ::= "0"|"1"|"2"|"3"|"4"|"5"|"6"|"7"|"8"|"9"
```

Una cadena valida es G 26 080190 B 22 062794 G 22 112194 B 13 010903.

2.1. Reconocer vs. generar

Toda gramatica define dos tareas duales:

- **Reconocedor:** dada una cadena, decide si pertenece al lenguaje (*parsear*). Existen herramientas (yacc/bison, ANTLR) y los programas las usan para validar inputs.
- **Generador:** dada la gramatica, deriva cadenas que estan en el lenguaje.

3. Criterios de cobertura sobre gramaticas

Definicion

Terminal Symbol Coverage (TSC). El conjunto de tests RT debe contener, para cada simbolo terminal t de la gramatica G , alguna cadena que lo use.

Definicion

Production Coverage (PDC). Para cada produccion p de la gramatica, RT debe contener alguna cadena cuya derivacion use p .

Idea clave

PDC subsume TSC: para usar todas las producciones es necesario haber usado tarde o temprano todos los simbolos terminales que aparecen en ellas. PDC es el criterio mas frecuente porque se mapea directo a las “reglas distintas” de la gramatica.

Criterios mas exigentes.

- **Derivation Coverage (DC).** Que cada *derivacion completa* posible aparezca en al-

guna cadena. En general infinita; se usa solo en gramaticas chicas o restringidas.

- **Cobertura de pares** de producciones, analoga a la cobertura de pares de aristas en grafos.

3.1. Cadenas mutadas vs. gramaticas mutadas

Cuando la cobertura no apunta a *validar* sino a *detectar manejo erroneo*, se generan cadenas *invalidas*. Hay dos formas:

- **Mutar la gramatica** (cambiar una regla) y generar a partir de la version mutada: las cadenas producidas *no* pertenecen al lenguaje original.
- **Mutar cadenas basicas** (validas), generando variantes que pueden quedar validas o invalidas dependiendo del cambio.

Idea clave

El esquema general es: “*partir de cadenas/derivaciones validas, aplicar mutaciones y ver si el sistema bajo prueba detecta o tolera el cambio*”. Si las mutaciones siempre producen cadenas invalidas, el oraculo es “el programa debe rechazar”. Si pueden quedar validas, el oraculo es “el programa debe seguir funcionando bien”.

4. Mutacion sobre programas

La aplicacion historica y mas popular del testing basado en sintaxis es **mutar el codigo** del programa bajo test.

Definicion

Programa mutante. Variante del programa P obtenida aplicando un *operador de mutacion* sobre una pequena porcion del codigo. Por ejemplo, cambiar `&&` por `||`, reemplazar `>` por `>=`, eliminar una asignacion, sustituir una constante por otra.

Definicion

Matar un mutante. Dado un mutante m del programa P y un test t , se dice que t mata a m si la salida de t sobre P difiere de la salida de t sobre m .

4.1. Tipos de mutantes

- **Mutante muerto:** hay al menos un test en la suite que lo distingue del original.

- **Mutante nacido muerto (*stillborn*):** sintacticamente invalido — el compilador lo rechaza. No aporta al score.
- **Mutante trivial:** casi cualquier test lo mata. Suelen ser baratos pero poco utiles porque no exigen mucho a la suite.
- **Mutante equivalente:** tiene exactamente el mismo comportamiento observable que el original. *No se puede matar* con ningun test. Hay que identificarlo manualmente y descontarlo del denominador del score.

Ejemplo

En `int abs(int x) { return x < 0 ? -x : x; }`, mutar `x < 0` a `x <= 0` da un mutante equivalente: la rama `-x` con $x = 0$ devuelve 0, igual que el original.

4.2. Mutation Score

Definicion

Mutation Score de una suite T sobre un conjunto de mutantes M :

$$\text{score}(T, M) = \frac{|\{m \in M : T \text{ mata } m\}|}{|M| - |\text{equivalentes}|}$$

El denominador se ajusta para no contar los mutantes equivalentes (que ningun test puede matar).

Idea clave

Un *mutation score* cercano al 100 % indica que la suite es sensible a *pequeñas perturbaciones* del codigo: hay algun input que las descubre. Es un indicador mas fuerte que cobertura de lineas o aristas, porque obliga al test a *notar el cambio en la salida*, no solo a ejecutar la linea.

4.3. Operadores tipicos

Para programas Java, herramientas como PIT incluyen operadores como:

- **Conditionals Boundary:** $< a \leq$, $> a \geq$, etc.
- **Negate Conditionals:** invertir la condicion de un `if`.
- **Math:** $+ \rightarrow -$, $* \rightarrow /$, etc.
- **Increments:** $i++$ a $i-$.

- **Return Values:** cambiar `return x` a `return 0` o `return null`.
- **Void Method Calls:** eliminar llamadas a metodos que retornan `void`.
- **Empty Returns, Primitive Returns, Null Returns,** etc.

4.4. Mutacion fuerte vs. debil

- **Strong mutation:** requiere que el cambio se observe *en la salida final* del programa. Es el modelo estandar y el que usa PIT.
- **Weak mutation:** basta con que en algun punto intermedio el estado del programa difiera del original (mas barato, pero da scores menos significativos).

4.5. Costo y por que vale la pena

Mutacion es caro: hay que ejecutar la suite una vez por cada mutante. Herramientas modernas (PIT) mitigan esto con:

- *Bytecode mutation:* muta el `.class` compilado, no el `.java`.
- *Test selection:* solo corre los tests que cubren la linea mutada.
- *Incremental analysis:* reusa resultados entre ejecuciones.

5. Fuzzing

5.1. Idea general

Fuzzing es generacion masiva de inputs (validos, invalidos o ambos) para alimentar a un programa y observar si crashea o se comporta mal. Es una tecnica orientada a robustez y seguridad mas que a verificacion de correccion funcional.

5.2. Random fuzzing

Definicion

Un **random fuzzer** genera cadenas *sin ninguna estructura previa*: elige una longitud al azar y rellena con caracteres aleatorios de un alfabeto dado.

Es facil de implementar pero rara vez logra que la entrada sea sintacticamente valida (para gramaticas no triviales). Por eso suele explorar mas el manejo de errores que el flujo “feliz”.

5.3. Mutation fuzzing

Definicion

Un **mutation fuzzer** parte de un conjunto de *semillas* (inputs validos conocidos) y aplica una secuencia de *mutaciones pequenas* para generar nuevos inputs. Cada mutacion puede borrar, insertar o modificar caracteres / tokens.

Operadores de mutacion sobre strings tipicos:

- `deleteRandomCharacter(s)`: elimina un caracter al azar.
- `insertRandomCharacter(s)`: inserta un caracter al azar.
- `flipRandomCharacter(s)`: cambia un caracter (por ejemplo flip de un bit).

Aplicar pocas mutaciones a una semilla valida produce inputs “casi validos” que ejercitan mucho mas codigo que strings totalmente aleatorios.

5.4. Random vs. mutation: el tradeoff

- **Random fuzzing** explora un espacio grande pero acierta poco. Bueno para detectar crashes en el lexer/parser.
- **Mutation fuzzing** explora cerca de los inputs validos. Bueno para detectar bugs en logica de negocio o partes profundas del codigo.

5.5. El oraculo en fuzzing

Un fuzzer genera miles de inputs; no se puede definir el resultado correcto para cada uno. El *oraculo* suele ser:

- **No crash**: el programa no debe terminar con `SIGABRT` (exit 134), `SIGSEGV` (exit 139), ni emitir mensajes de error catastroficos en `stderr` (“segmentation fault”, “core dumped”, “illegal instruction”).
- **Sin assertion violations** (en codigo con `assert`).
- **Sin uso indebido** de memoria (detectable con sanitizers).

Idea clave

Es importante que el oraculo *no* exija que la salida sea “correcta” en sentido funcional: la mayoría de los inputs fuzzed son invalidos, y la respuesta correcta es un error controlado. Lo que el fuzzer busca es *fallas que el programa no maneja*.

5.6. Seguridad

El *warning* del enunciado es real: si se conecta un fuzzer a un programa que puede *modificar el sistema* (e.g. `rm`, `dd`, un script con permisos), se corre el riesgo de que alguna entrada generada provoque dano real. Por eso se eligen programas *seguros* para fuzzear (un parser, una calculadora como `bc`, un sandbox, etc.).

6. Pitest en la practica

6.1. Que hace

Pitest es una herramienta de testing de mutacion para Java. Trabaja sobre el bytecode compilado, lo cual le da velocidad y soporte para los grandes operadores de mutacion sin modificar el fuente.

6.2. Configuracion tipica (pom.xml)

```
<plugin>
  <groupId>org.pitest</groupId>
  <artifactId>pitest-maven</artifactId>
  <version>1.8.0</version>
  <configuration>
    <targetClasses><param>my.pkg.MyClass</param></targetClasses>
    <targetTests><param>my.pkg.MyClassTests</param></targetTests>
  </configuration>
  <dependencies>
    <dependency>
      <groupId>org.pitest</groupId>
      <artifactId>pitest-junit5-plugin</artifactId>
      <version>0.16</version>
    </dependency>
  </dependencies>
</plugin>
```

6.3. Ejecucion

```
mvn test org.pitest:pitest-maven:mutationCoverage
```

PIT genera un reporte HTML con: total de mutantes, cuantos KILLED, SURVIVED, NO_COVERAGE, TIMED_OUT, MEMORY_ERROR, etc. Para apuntar a una clase distinta sin tocar el `pom.xml` se sobrescribe en linea de comando con `-DtargetClasses` y `-DtargetTests`.

6.4. Como leer el reporte

- **KILLED:** el test que lo mato esta indicado.
- **SURVIVED:** hay que analizarlo — o agregar un test que lo distinga, o justificar que es equivalente.
- **NO_COVERAGE:** ningun test ejecuta esa linea; antes de mutacion, conviene primero cubrir.
- **TIMED_OUT:** el mutante introdujo un bucle infinito; cuenta como muerto.

7. Comparacion con criterios anteriores

Criterio	Modelo	Pregunta clave
Particion del input	dominio	¿cubri todas las clases de equivalencia?
NC, EC, EPC, PPC	grafo (CFG)	¿recorri todos los nodos/aristas?
PC, CC, CACC, RACC	predicados logicos	¿cada clausula importa?
TSC, PDC	gramatica	¿use todas las producciones?
Mutation Score	programa mutado	¿mis tests notan cambios pequenos?
Fuzzing	generacion masiva	¿hay alguna entrada que crashee?

Idea clave

La cobertura de mutacion *subsume* en la practica a casi todos los criterios anteriores: para matar mutantes hace falta haber recorrido las lineas (NC), las aristas (EC) y haber hecho que las decisiones tomen distintos valores (PC/CC). No es una equivalencia formal, pero empiricamente las suites con mutation score alto suelen tener cobertura estructural alta.

8. Glosario

- **Operador de mutacion:** regla que produce un mutante a partir del programa original cambiando un constructo pequeno.
- **Mutante equivalente:** variante con comportamiento observable identico al original.
- **Mutation Score:** fraccion de mutantes muertos, descontando equivalentes.
- **Seed (semilla):** input valido que se usa como punto de partida del mutation fuzzing.
- **Oraculo:** criterio que decide si una corrida es “correcta” o “fallida”. En fuzzing suele reducirse a “no crash”.

- **Stillborn:** mutante sintacticamente invalido.

Conclusion

El testing basado en sintaxis es un puente entre la estructura formal de los artefactos (gramaticas, codigo fuente) y la generacion automatica de tests. La mutacion de programas proporciona un criterio fuerte de calidad de suite: una suite con *mutation score* alto no solo recorre el codigo, sino que es *sensible* a cambios. El fuzzing, por su parte, escala la generacion de inputs a miles de casos y se apoya en oraculos minimos para detectar fallas graves. Las tres tecnicas son complementarias: la mutacion mide la suite, el fuzzing estresa el sistema, y la cobertura sintactica garantiza que ningun rincon de la gramatica de inputs quede sin tocar.